

# Trees & BST in C++ From One Node to Placement-Ready

---

Everything you need for binary trees and binary search trees: the recursion mental model, all six traversals, the 14 problem patterns, an approach framework, the bug catalogue, and a curated problem list.

**43**

SECTIONS

**14**

PATTERNS

**50**

CODE BLOCKS

**38**

PROBLEMS

---

Prepared for **Hari** · on-campus placement preparation  
Companion to **Trees-and-BST-Solved-Questions.pdf**  
Generated 07 September 2026

# Contents

## PART 0 — Trees, from absolute zero

---

- 0.1 One pointer becomes two
- 0.2 The vocabulary you must be fluent in
- 0.3 The shapes, and why they decide your complexity
- 0.4 The node in C++, and building one by hand
- 0.5 How LeetCode hands you a tree
- 0.6 The call stack — where recursion actually lives
- 0.7 Deleting a tree (and why C++ interviewers ask)

## PART 1 — The one mental model for tree recursion

---

- 1.1 "Handle this node, and trust the children"
- 1.2 The base case is nullptr, essentially always
- 1.3 Top-down versus bottom-up — the decision that shapes your code
- 1.4 The most valuable trick: return one thing, record another
- 1.5 Designing the return type

## PART 2 — The traversals

---

- 2.1 The three depth-first orders
- 2.2 What each order is actually for
- 2.3 Iterative inorder — the one to memorise
- 2.4 Iterative preorder and postorder
- 2.5 Level-order (BFS) and the level-size trick
- 2.6 Morris traversal —  $O(1)$  space

## PART 3 — The Binary Search Tree

---

- 3.1 The property — stated correctly
- 3.2 Search, insert, delete
- 3.3 The master key: inorder gives sorted order
- 3.4 Validating a BST — the range method
- 3.5 Balance, and what to say about AVL / Red-Black

## PART 4 — The complete pattern taxonomy

---

- Pattern 1 — Traversal & collection

Pattern 2 — Aggregate over the whole tree

Pattern 3 — Return one thing, record another

Pattern 4 — Root-to-leaf paths

Pattern 5 — The level-order family

Pattern 6 — Two-tree comparison and structural edits

Pattern 7 — Lowest common ancestor

Pattern 8 — Construction from traversals

Pattern 9 — BST operations

Pattern 10 — BST plus inorder ordering

Pattern 11 — Serialisation and subtree encoding

Pattern 12 — Treating the tree as a graph

Pattern 13 — In-place restructuring

Pattern 14 — Counting and DP on trees

## **PART 5 — How to approach ANY tree problem**

---

Step 1 — Draw a tree with 5-7 nodes, and make it lopsided

Step 2 — Name the pattern

Step 3 — Decide top-down or bottom-up

Step 4 — Write the base case first, literally first

Step 5 — Combine, then check the leaf case separately

Step 6 — Dry-run four inputs before you submit

## **PART 6 — The bug catalogue**

---

## **PART 7 — Complexity cheat sheet**

---

## **PART 8 — The problem list**

---

## **PART 9 — A four-week plan**

---

## **Quick reference card**

---

# PART 0 — Trees, from absolute zero

---

You already know a linked list: a node with a value and **one** `next` pointer, so the data forms a straight line. A binary tree changes exactly one thing — the node gets **two** pointers instead of one. That single change turns a line into a branching structure, and everything else in this guide follows from it.

## 0.1 One pointer becomes two

```
struct ListNode {                // a line
    int val;
    ListNode* next;
};

struct TreeNode {               // a branching structure
    int val;
    TreeNode* left;
    TreeNode* right;
};
```

That is the whole structural difference. A list node says "the rest of the data is over there." A tree node says "part of the rest is over there, and another part is over *there*."

LINKED LIST

1 -> 2 -> 3 -> 4

BINARY TREE



Because the structure branches, you can no longer walk it with a single `while` loop. You need either **recursion** (Part 1) or an explicit **stack/queue** (Part 2). That is the only real difficulty in this topic, and once it clicks, tree problems become the most mechanical category in all of DSA.

## 0.2 The vocabulary you must be fluent in

Interviewers use these words without explaining them. Learn them once, properly.

| TERM                      | MEANING                                                                                                         | IN THE TREE ABOVE           |
|---------------------------|-----------------------------------------------------------------------------------------------------------------|-----------------------------|
| <b>Root</b>               | The single node with no parent — the entry point                                                                | 1                           |
| <b>Leaf</b>               | A node with <b>no children at all</b> (both <code>left</code> and <code>right</code> are <code>nullptr</code> ) | 4, 5, 6                     |
| <b>Internal node</b>      | Any non-leaf node                                                                                               | 1, 2, 3                     |
| <b>Parent / child</b>     | 2 is the parent of 4; 4 is a child of 2                                                                         | -                           |
| <b>Sibling</b>            | Two nodes with the same parent                                                                                  | 4 and 5                     |
| <b>Ancestor</b>           | Any node on the path from the root down to it                                                                   | ancestors of 5 are 2, 1     |
| <b>Descendant</b>         | Anything reachable by going downward                                                                            | descendants of 2 are 4, 5   |
| <b>Edge</b>               | One parent→child link                                                                                           | 1→2 is an edge              |
| <b>Subtree</b>            | A node <b>plus everything below it</b> — itself a valid tree                                                    | the subtree at 2 is 2, 4, 5 |
| <b>Level / depth</b>      | Number of edges from the root down to the node (root is depth 0)                                                | 4 is at depth 2             |
| <b>Height of a node</b>   | Number of edges on the <b>longest</b> downward path to a leaf                                                   | height of 2 is 1            |
| <b>Height of the tree</b> | Height of the root                                                                                              | 2                           |

**The one that trips everyone up:** *depth* is measured downward **from the root**, *height* is measured upward **from the deepest leaf**. Depth is known on the way down; height is only known on the way back up. That single fact decides whether a problem is solved top-down or bottom-up — see Part 1.4.

A note on the leaf definition, because it causes real bugs: a node with a left child but no right child is **not** a leaf. Problems like *Minimum Depth* (LC 111) and *Path Sum* (LC 112) are wrong on half of all submissions purely because people write `if (!node->left && !node->right)` incorrectly, or forget it entirely.

### 0.3 The shapes, and why they decide your complexity

PERFECT



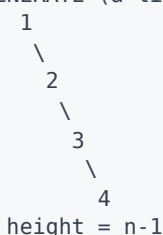
height =  $\log n$

COMPLETE



height =  $\log n$

DEGENERATE (a list!)



height =  $n-1$

| SHAPE                      | DEFINITION                                                                          | HEIGHT                   |
|----------------------------|-------------------------------------------------------------------------------------|--------------------------|
| <b>Full</b>                | Every node has 0 or 2 children (never exactly 1)                                    | varies                   |
| <b>Complete</b>            | Every level filled except possibly the last, which fills left to right              | $\lceil \log_2 n \rceil$ |
| <b>Perfect</b>             | All internal nodes have 2 children, all leaves on the same level                    | $\log_2 (n+1) - 1$       |
| <b>Balanced</b>            | For every node, $ \text{height}(\text{left}) - \text{height}(\text{right})  \leq 1$ | $O(\log n)$              |
| <b>Degenerate / skewed</b> | Every node has one child — effectively a linked list                                | $n - 1$                  |

This table is the reason almost every tree answer contains the phrase "*O(h)*, where *h* is the height — *O(log n)* if balanced, *O(n)* in the worst case." Say that sentence in interviews. It shows you know the tree could be a stick.

A perfect tree of height  $h$  has  $2^{(h+1)} - 1$  nodes, and  $2^h$  of them are leaves — roughly **half of all nodes in a tree are leaves**. That is worth remembering when you reason about recursion cost.

## 0.4 The node in C++, and building one by hand

This is the exact definition LeetCode gives you:

```
struct TreeNode {
    int val;
    TreeNode *left;
    TreeNode *right;
    TreeNode() : val(0), left(nullptr), right(nullptr) {}
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
    TreeNode(int x, TreeNode *left, TreeNode *right)
        : val(x), left(left), right(right) {}
};
```

Building the tree from 0.1 by hand, so you can see there is no magic:

```
TreeNode* root = new TreeNode(1);
root->left = new TreeNode(2);
root->right = new TreeNode(3);
root->left->left = new TreeNode(4);
root->left->right = new TreeNode(5);
root->right->right = new TreeNode(6);

// reading a value three levels down:
cout << root->left->right->val;    // 5
```

Every one of those `new` calls returns a heap address. `root` holds one address; the tree exists only as a web of addresses. `nullptr` means "there is no child here" — exactly as it meant "the list ends here" for a linked list.

**Dereference discipline, carried over from linked lists:** never write `node->left->val` without knowing `node->left` is not null. In tree code the null check is almost always the recursive base case, so this bug shows up as a *missing base case*.

## 0.5 How LeetCode hands you a tree

You will see input written like this:

```
root = [3,9,20,null,null,15,7]
```

That is **level-order (breadth-first)** with **null** placeholders. Read it left to right, filling the tree level by level:

```
      3
     / \
    9   20
   / \  / \
  15 7 15 7

[3, 9, 20, null, null, 15, 7]
3 -> children 9, 20
9 -> children null, null
20 -> children 15, 7
```

You never have to parse this in a submission — LeetCode builds the tree for you. But you *do* need to read it fluently to check your own work, and you need to build such trees yourself when testing locally. Here is the builder used throughout the companion solutions file:

```
// Build a tree from LeetCode's level-order array. Use INT_MIN for null.
TreeNode* build(vector<int> a) {
    if (a.empty() || a[0] == INT_MIN) return nullptr;
    TreeNode* root = new TreeNode(a[0]);
    queue<TreeNode*> q; q.push(root);
    size_t i = 1;
    while (!q.empty() && i < a.size()) {
        TreeNode* cur = q.front(); q.pop();
        if (i < a.size() && a[i] != INT_MIN) { cur->left = new TreeNode(a[i]); q.push(cur->left); }
        i++;
        if (i < a.size() && a[i] != INT_MIN) { cur->right = new TreeNode(a[i]); q.push(cur->right); }
        i++;
    }
    return root;
}
```

Note what this tells you about the format: **nulls are only listed for nodes whose parent exists**. A null child never gets two null grandchildren listed after it.

## 0.6 The call stack — where recursion actually lives

When you call a function, the machine pushes a **stack frame** holding its parameters and locals, and pops it on return. Recursion just means many frames of the same function are alive at once.

```
int height(TreeNode* n) {
    if (!n) return -1; // frame returns immediately
    return 1 + max(height(n->left), height(n->right)); // two frames pushed, one at a time
}
```

For the tree in 0.1, the stack at its deepest looks like:

```
height(node 4)  <- top, returns -1 then 0
height(node 2)
height(node 1)  <- bottom
```

Two consequences you must be able to state:

- **Space complexity of any recursive tree function is  $O(h)$**  — one frame per level on the current path, not  $O(n)$ . For a balanced tree that is  $O(\log n)$ ; for a skewed tree it degrades to  $O(n)$ .
- **Deep recursion can stack-overflow.** With  $n$  up to  $10^5$  and a skewed tree, recursion depth is  $10^5$  frames. It usually survives on LeetCode, but this is exactly why iterative traversals (Part 2.3) exist and why interviewers ask for them.

## 0.7 Deleting a tree (and why C++ interviewers ask)

```
void destroy(TreeNode* root) {  
    if (!root) return;  
    destroy(root->left);  
    destroy(root->right);  
    delete root;           // children first, THEN the parent  
}
```

The order is the point. If you `delete root` first, `root->left` reads freed memory and you have undefined behaviour. This is **postorder** traversal, and it is the canonical example of *why postorder exists*: any operation that must finish with the children before it touches the parent is postorder.

# PART 1 — The one mental model for tree recursion

---

Almost every tree problem is solved by the same three-line skeleton. If you internalise this part, you will stop memorising solutions and start deriving them.

## 1.1 "Handle this node, and trust the children"

Recursion on trees works because **a subtree is a tree**. `root->left` is not a special reduced case — it is a complete, valid tree in its own right. So the function you are writing already works on it, by assumption. Your only job is:

1. Say what to do when the tree is empty (the **base case**).
2. Assume the answers for `left` and `right` are already correct.
3. Combine them with the current node's value into this node's answer.

```
ReturnType solve(TreeNode* node) {
    if (!node) return <answer for an empty tree>; // 1

    ReturnType L = solve(node->left);           // 2 - trust it
    ReturnType R = solve(node->right);          // 2 - trust it

    return <combine L, R and node->val>;        // 3
}
```

Do not trace the recursion in your head past one level. It works or it does not, and it works precisely when steps 1 and 3 are right for a single node. Tracing three levels deep is how people confuse themselves out of a correct solution.

Worked instance — maximum depth:

```
int maxDepth(TreeNode* root) {
    if (!root) return 0; // empty tree: depth 0
    return 1 + max(maxDepth(root->left), maxDepth(root->right)); // me + the deeper child
}
```

Every line maps to a step. That is all `maxDepth` ever is.

## 1.2 The base case is `nullptr`, essentially always

Beginners write the base case as "if this is a leaf." Prefer `if (!node)`. Two reasons:

- A node with exactly one child is not a leaf, so leaf-based base cases silently skip it.
- Checking `!node` means the caller never has to check before recursing, which removes half the null tests in the body.

```
// fragile
if (!root->left && !root->right) return 1; // crashes when root is null to begin with

// robust
if (!root) return 0;
```

The exception is genuinely leaf-specific logic — root-to-**leaf** path problems (LC 112, 113, 129, 111). There you need *both*: `if (!node) return ...;` to guard, and a separate `if (!node->left && !node->right)` to record the answer. Part 4, Pattern 4 covers this in full.

## 1.3 Top-down versus bottom-up — the decision that shapes your code

This is the single most useful distinction in tree problems.

|                   | TOP-DOWN                                           | BOTTOM-UP                                           |
|-------------------|----------------------------------------------------|-----------------------------------------------------|
| Information flows | Root → leaves, carried in parameters               | Leaves → root, carried in return values             |
| Looks like        | <code>void dfs(node, accumulated)</code>           | <code>T dfs(node)</code> returning a computed value |
| Answer lives in   | An outer variable / passed reference               | The return value of the root call                   |
| Natural for       | Paths, prefixes, running sums, depth, valid ranges | Height, size, diameter, "is this subtree ok"        |
| Traversal order   | Preorder                                           | Postorder                                           |

Top-down, carrying the running state down:

```
void dfs(TreeNode* node, int sumSoFar, int& best) {
    if (!node) return;
    sumSoFar += node->val;
    if (!node->left && !node->right) best = max(best, sumSoFar);
    dfs(node->left, sumSoFar, best);
    dfs(node->right, sumSoFar, best);
}
```

Bottom-up, returning the computed value up:

```
int height(TreeNode* node) {
    if (!node) return 0;
    return 1 + max(height(node->left), height(node->right));
}
```

**How to choose:** ask "can I answer for this node using only what is below it?" If yes, go bottom-up. If you need to know where you came from — the path so far, the running sum, the allowed value range — go top-down.

## 1.4 The most valuable trick: return one thing, record another

A large family of "hard" tree problems is solved by a function that **returns** the value the parent needs while **recording** the global answer in a captured variable on the way. Diameter, balanced-check, and maximum path sum are all the same trick.

Take **diameter** (LC 543). The longest path through a given node is `height(left) + height(right)`. But the parent does not want that number — the parent wants the height. So return the height and record the diameter:

```

int best = 0;

int height(TreeNode* n) {
    if (!n) return 0;
    int L = height(n->left);
    int R = height(n->right);
    best = max(best, L + R);    // RECORD: answer for a path bending at n
    return 1 + max(L, R);      // RETURN: what my parent needs
}

```

One postorder pass,  $O(n)$ . The naive alternative — calling `height()` inside a `diameter()` recursion — is  $O(n^2)$  and is a very common interview rejection.

Say this out loud until it is automatic: **"I return the height, and I record the diameter."** Then apply the same sentence shape to:

| PROBLEM                   | RETURN TO PARENT                  | RECORD GLOBALLY                   |
|---------------------------|-----------------------------------|-----------------------------------|
| 543 Diameter              | height                            | <code>L + R</code>                |
| 110 Balanced              | height, or $-1$ as a poison value | (or a bool flag)                  |
| 124 Max Path Sum          | best downward path from here      | <code>L + R + node-&gt;val</code> |
| 250 Count Unival Subtrees | is this subtree unival            | count                             |
| 1372 Longest ZigZag       | (left-len, right-len)             | max seen                          |

## 1.5 Designing the return type

When one value is not enough, return a small struct. This is cleaner and far less bug-prone than juggling three reference parameters, and interviewers like it.

```

struct Info {
    bool isBST;
    int mn, mx;    // min and max in this subtree
    int size;      // size of the largest BST inside it
};

Info dfs(TreeNode* node) {
    if (!node) return {true, INT_MAX, INT_MIN, 0};
    Info L = dfs(node->left), R = dfs(node->right);
    bool ok = L.isBST && R.isBST && L.mx < node->val && node->val < R.mn;
    ...
}

```

That pattern — *bundle everything the parent needs into one struct and compute it in one postorder pass* — solves "largest BST subtree", "is this a valid BST" without ranges, and several GfG favourites in a single traversal.

# PART 2 — The traversals

There are exactly four traversal orders you must know cold, plus one space-optimised curiosity. Everything in Part 4 is built on them.

## 2.1 The three depth-first orders

The names describe **when you touch the current node relative to its children**:

```
void preorder(TreeNode* n, vector<int>& out) {    // NODE, left, right
    if (!n) return;
    out.push_back(n->val);
    preorder(n->left, out);
    preorder(n->right, out);
}

void inorder(TreeNode* n, vector<int>& out) {    // left, NODE, right
    if (!n) return;
    inorder(n->left, out);
    out.push_back(n->val);
    inorder(n->right, out);
}

void postorder(TreeNode* n, vector<int>& out) {    // left, right, NODE
    if (!n) return;
    postorder(n->left, out);
    postorder(n->right, out);
    out.push_back(n->val);
}
```

The three functions are the same three lines in three different orders. On this tree:



|              |             |                            |
|--------------|-------------|----------------------------|
| preorder     | : 1 2 4 5 3 | (root first – top-down)    |
| inorder      | : 4 2 5 1 3 | (sorted, if this is a BST) |
| postorder    | : 4 5 2 3 1 | (root last – bottom-up)    |
| level-order: | 1 2 3 4 5   | (row by row)               |

## 2.2 What each order is actually for

Do not memorise the outputs. Memorise the **use**:

| ORDER              | USE IT WHEN                                                                                               | TYPICAL PROBLEMS                                            |
|--------------------|-----------------------------------------------------------------------------------------------------------|-------------------------------------------------------------|
| <b>Preorder</b>    | You must process a node <b>before</b> its children — copying a tree, serialising, carrying state downward | 144, 297 serialize, 257 all paths, 114 flatten              |
| <b>Inorder</b>     | The tree is a <b>BST</b> and you want sorted order                                                        | 94, 98 validate, 230 kth smallest, 99 recover, 173 iterator |
| <b>Postorder</b>   | The parent's answer <b>depends on the children's</b> — heights, sizes, deletion, aggregation              | 145, 104, 110, 543, 124, 337                                |
| <b>Level-order</b> | The problem mentions levels, rows, depth-by-depth, "closest", or views                                    | 102, 103, 107, 199, 111, 662, 116                           |

**The single most useful fact in this whole guide: the inorder traversal of a BST is the sorted list of its values.** At least five of the problems in Part 8 are one line of code once you see that.

## 2.3 Iterative inorder — the one to memorise

Interviewers ask for iterative traversal specifically to see whether you understand the call stack. Inorder is the important one; learn this exact shape:

```
vector<int> inorderIterative(TreeNode* root) {
    vector<int> out;
    stack<TreeNode*> st;
    TreeNode* cur = root;
    while (cur || !st.empty()) {
        while (cur) { st.push(cur); cur = cur->left; } // dive left, remembering the way back
        cur = st.top(); st.pop();
        out.push_back(cur->val); // visit
        cur = cur->right; // then handle the right subtree
    }
    return out;
}
```

Read it as: *go as far left as you can, pushing every node; pop, visit, then step right and repeat.* The stack is doing by hand exactly what the recursion did for you. Loop condition `cur || !st.empty()` matters: at the moment you step right into a null child, the stack still holds unfinished ancestors.

This same skeleton, paused between iterations, **is** the BST Iterator (LC 173).

## 2.4 Iterative preorder and postorder

Preorder is easy — a stack, right child pushed first so the left is processed first:

```
vector<int> preorderIterative(TreeNode* root) {
    vector<int> out;
    if (!root) return out;
    stack<TreeNode*> st; st.push(root);
    while (!st.empty()) {
        TreeNode* n = st.top(); st.pop();
        out.push_back(n->val);
        if (n->right) st.push(n->right);    // right first...
        if (n->left) st.push(n->left);      // ...so left pops first
    }
    return out;
}
```

Postorder has a trick worth knowing: do a preorder with the children pushed the *other* way — giving node, right, left — then reverse it.

```
vector<int> postorderIterative(TreeNode* root) {
    vector<int> out;
    if (!root) return out;
    stack<TreeNode*> st; st.push(root);
    while (!st.empty()) {
        TreeNode* n = st.top(); st.pop();
        out.push_back(n->val);
        if (n->left) st.push(n->left);
        if (n->right) st.push(n->right);
    }
    reverse(out.begin(), out.end());    // node,right,left reversed = left,right,node
    return out;
}
```

## 2.5 Level-order (BFS) and the level-size trick

Level-order uses a **queue**, not a stack. The one thing that makes it useful is capturing `q.size()` **before** the inner loop — that count is exactly the number of nodes on the current level, so you can group by level.

```
vector<vector<int>> levelOrder(TreeNode* root) {
    vector<vector<int>> out;
    if (!root) return out;
    queue<TreeNode*> q; q.push(root);
    while (!q.empty()) {
        int sz = q.size();    // freeze the level width HERE
        vector<int> level;
        for (int i = 0; i < sz; i++) {
            TreeNode* n = q.front(); q.pop();
            level.push_back(n->val);
            if (n->left) q.push(n->left);
            if (n->right) q.push(n->right);
        }
        out.push_back(level);
    }
    return out;
}
```

If you read `q.size()` inside the loop instead, you mix in the children you just pushed and the levels smear together. That is bug #6 in Part 6.

This one template, with tiny edits, solves an entire family:

| CHANGE                                  | PROBLEM                              |
|-----------------------------------------|--------------------------------------|
| take <code>level.back()</code>          | 199 Right Side View                  |
| take <code>level.front()</code>         | left side view                       |
| reverse alternate levels                | 103 Zigzag                           |
| <code>reverse(out)</code> at the end    | 107 Level Order II                   |
| average of <code>level</code>           | 637 Average of Levels                |
| stop at the first leaf                  | 111 Minimum Depth (fastest solution) |
| track index positions                   | 662 Maximum Width                    |
| link <code>q.front()</code> to the next | 116 / 117 Next Right Pointers        |

## 2.6 Morris traversal — $O(1)$ space

Occasionally an interviewer asks for inorder traversal in constant space. The answer is Morris traversal: temporarily rewire the rightmost node of the left subtree to point back at the current node (a *thread*), walk down, and undo the thread when you come back.

```
vector<int> morrisInorder(TreeNode* root) {
    vector<int> out;
    TreeNode* cur = root;
    while (cur) {
        if (!cur->left) { out.push_back(cur->val); cur = cur->right; }
        else {
            TreeNode* pred = cur->left;
            while (pred->right && pred->right != cur) pred = pred->right;
            if (!pred->right) { pred->right = cur; cur = cur->left; } // thread it
            else { pred->right = nullptr; out.push_back(cur->val); cur = cur->right; } // unthread
        }
    }
    return out;
}
```

Know that it exists, know it is  $O(n)$  time and  $O(1)$  space, and know the cost: it mutates the tree during the walk. It is a bonus answer, not a first answer.

# PART 3 — The Binary Search Tree

A BST is a binary tree with one extra rule, and that rule turns  $O(n)$  searching into  $O(h)$ .

## 3.1 The property — stated correctly

For **every** node:

- all values in its **entire left subtree** are **less than** the node's value, and
- all values in its **entire right subtree** are **greater than** the node's value.



**The classic mistake:** thinking the rule is only about a node's immediate children. It is about the whole subtree. This tree is **not** a BST even though every parent-child pair looks fine:

`10 / \ 5 15 / \ 6 20 <- 6 is in 10's RIGHT subtree but 6 < 10`

This exact shape is the counterexample LC 98 uses to fail naive solutions.

Duplicates: LeetCode BST problems assume all values are distinct unless stated. If asked in an interview, say you would put duplicates consistently on one side, or store a count in the node.

## 3.2 Search, insert, delete

**Search** — compare and descend.  $O(h)$ .

```
TreeNode* search(TreeNode* root, int key) {
    while (root && root->val != key)
        root = key < root->val ? root->left : root->right;
    return root;
    // nullptr if absent
}
```

**Insert** — descend to the empty slot where the key belongs and hang a new node there. The recursive version returning a subtree pointer is the cleanest form, and the "reassign the child from the return value" idiom is worth learning because delete uses it too:

```
TreeNode* insert(TreeNode* root, int val) {
    if (!root) return new TreeNode(val);
    if (val < root->val) root->left = insert(root->left, val);
    else root->right = insert(root->right, val);
    return root;
}
```

**Delete** — the only fiddly BST operation, and a very common interview question. Three cases:

- |                                         |                                                    |                                                                                                                                   |
|-----------------------------------------|----------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| (a) leaf<br>delete it<br>return nullptr | (b) one child<br>splice the child<br>up, return it | (c) two children<br>replace value with the<br>INORDER SUCCESSOR (smallest<br>in the right subtree), then<br>delete that successor |
|-----------------------------------------|----------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|

```
TreeNode* deleteNode(TreeNode* root, int key) {
    if (!root) return nullptr;
    if (key < root->val) root->left = deleteNode(root->left, key);
    else if (key > root->val) root->right = deleteNode(root->right, key);
    else {
        if (!root->left) return root->right; // covers (a) and (b)
        if (!root->right) return root->left; // covers (b)
        TreeNode* succ = root->right; // (c)
        while (succ->left) succ = succ->left; // smallest in right subtree
        root->val = succ->val;
        root->right = deleteNode(root->right, succ->val);
    }
    return root;
}
```

Case (c) works because the inorder successor is the **next value in sorted order**, so putting it at the node keeps every ordering constraint intact. Using the inorder *predecessor* (largest in the left subtree) is equally valid — say so, it shows you understand why.

### 3.3 The master key: inorder gives sorted order

Because  $\text{left} < \text{node} < \text{right}$  holds everywhere, walking left-node-right visits values in ascending order. Consequences you can use immediately:

| QUESTION                                      | ANSWER VIA INORDER                                          |
|-----------------------------------------------|-------------------------------------------------------------|
| Is this a valid BST?                          | is the inorder strictly increasing?                         |
| k-th smallest value                           | the k-th element emitted — stop early                       |
| Minimum / maximum                             | leftmost / rightmost node — no traversal needed, just walk  |
| Two nodes swapped by mistake                  | find the two inversions in the inorder sequence (LC 99)     |
| Convert BST to sorted list / greater-sum tree | inorder, or <b>reverse</b> inorder for suffix sums (LC 538) |
| Closest value to a target                     | track the best while descending                             |

The "stop early" version of k-th smallest is worth writing once:

```
int k_, ans_;
void inorderKth(TreeNode* n) {
    if (!n || k_ == 0) return;
    inorderKth(n->left);
    if (--k_ == 0) { ans_ = n->val; return; }
    inorderKth(n->right);
}
```

### 3.4 Validating a BST — the range method

The correct general solution passes down the **open interval** each subtree is allowed to live in. Note it is top-down: information flows from the root outward.

```
bool valid(TreeNode* n, long lo, long hi) {
    if (!n) return true;
    if (n->val <= lo || n->val >= hi) return false;
    return valid(n->left, lo, n->val) && valid(n->right, n->val, hi);
}
bool isValidBST(TreeNode* root) { return valid(root, LONG_MIN, LONG_MAX); }
```

Going left tightens the upper bound to the current value; going right tightens the lower bound. Use `long` (or pass pointers) because a node may legitimately hold `INT_MIN` or `INT_MAX` and `int` sentinels would then produce a false negative — a favourite hidden test case.

### 3.5 Balance, and what to say about AVL / Red-Black

Every BST operation costs  $O(h)$ . Inserting sorted data into a plain BST produces the degenerate stick from Part 0.3, and  $O(h)$  becomes  $O(n)$  — the data structure has silently degraded into a linked list.

**Self-balancing** trees fix this by rotating after insertions and deletions to keep  $h$  at  $O(\log n)$ .

|                 | AVL                                      | RED-BLACK                                                               |
|-----------------|------------------------------------------|-------------------------------------------------------------------------|
| Balance rule    | height difference $\leq 1$ at every node | colour rules bound the longest path at $2\times$ the shortest           |
| Balance         | stricter                                 | looser                                                                  |
| Lookups         | faster                                   | slightly slower                                                         |
| Insert / delete | more rotations                           | fewer rotations                                                         |
| Used by         | read-heavy structures, databases         | <code>std::map</code> , <code>std::set</code> , most language libraries |

For on-campus interviews you generally need: what a rotation is (a local re-pointing that preserves inorder order), why balancing exists, and the fact that C++'s `std::map` / `std::set` are red-black trees with  $O(\log n)$  operations. You will very rarely be asked to *code* AVL rotations; you will quite often be asked what a rotation does.

right rotation about y



inorder order is unchanged:

A x B y C    before  
A x B y C    after

# PART 4 — The complete pattern taxonomy

Tree questions look endless, but they are drawn from **14 patterns**. Every problem in Part 8 is labelled with the pattern it belongs to. When you recognise the pattern, you already have most of the code.

| #  | PATTERN                                | RECOGNISE IT BY                                         |
|----|----------------------------------------|---------------------------------------------------------|
| 1  | Traversal & collection                 | "return the traversal", "collect all values"            |
| 2  | Aggregate over the whole tree          | depth, size, sum, count of something                    |
| 3  | Return one thing, record another       | diameter, balanced, max path sum                        |
| 4  | Root-to-leaf paths                     | "path", "leaf", "path sum", "all paths"                 |
| 5  | Level-order / BFS family               | "level", "row", "view", "width", "closest"              |
| 6  | Two-tree comparison & structural edits | same, symmetric, subtree, invert, merge                 |
| 7  | Lowest common ancestor                 | "lowest common ancestor", "deepest node containing"     |
| 8  | Construction from traversals           | "build the tree from ..."                               |
| 9  | BST operations                         | search / insert / delete / floor / ceil / range         |
| 10 | BST + inorder ordering                 | kth smallest, validate, recover, greater-sum            |
| 11 | Serialisation & encoding               | "serialize", "encode as a string", "duplicate subtrees" |
| 12 | Tree as a graph                        | "distance K", "parent pointers", "burn the tree"        |
| 13 | In-place restructuring                 | flatten, connect next pointers, threading               |
| 14 | Counting / DP on trees                 | house robber III, count good nodes, unique BSTs         |

## Pattern 1 — Traversal & collection

**Shape:** walk the whole tree, push values into a container. Nothing is computed; the traversal *is* the answer.

```
void dfs(TreeNode* n, vector<int>& out) {  
    if (!n) return;  
    out.push_back(n->val);           // move this line to change pre/in/post  
    dfs(n->left, out);  
    dfs(n->right, out);  
}
```

**Problems:** 144, 94, 145, 589/590 (n-ary), 872 Leaf-Similar Trees, 897 Increasing Order Search Tree.

**Interview follow-up, always:** "now do it iteratively." Have Part 2.3 ready.

## Pattern 2 — Aggregate over the whole tree

**Shape:** one number that folds the children's numbers together. Pure bottom-up, base case is the identity element (0 for sums and counts, `-1` or `0` for heights, `INT_MIN` for maxima).

```
int count(TreeNode* n) {                // size
    if (!n) return 0;
    return 1 + count(n->left) + count(n->right);
}
int sum(TreeNode* n) {                  // sum of all values
    if (!n) return 0;
    return n->val + sum(n->left) + sum(n->right);
}
int maxDepth(TreeNode* n) {             // height
    if (!n) return 0;
    return 1 + max(maxDepth(n->left), maxDepth(n->right));
}
```

**Problems:** 104 Maximum Depth, 111 Minimum Depth (careful — see below), 222 Count Complete Tree Nodes, 404 Sum of Left Leaves, 559 N-ary Max Depth.

**111 Minimum Depth is not the mirror of 104.** `min(left, right)` is wrong when one child is missing: a node with only a right child would report depth 1 through its null left side, but that is not a root-to-leaf path. You must special-case the single-child node:

```
cpp int minDepth(TreeNode* n) { if (!n) return 0; if (!n->left) return 1 + minDepth(n->right); // only one real path if (!n->right) return 1 + minDepth(n->left); return 1 + min(minDepth(n->left), minDepth(n->right)); }
```

## Pattern 3 — Return one thing, record another

Covered in Part 1.4 — the highest-value pattern in this guide. The parent needs a *different* number than the answer being computed, so return the parent's number and keep the global answer in a captured variable.

```
class Solution {
    int best = 0;
    int depth(TreeNode* n) {
        if (!n) return 0;
        int L = depth(n->left), R = depth(n->right);
        best = max(best, L + R);           // RECORD: path bending at n
        return 1 + max(L, R);             // RETURN: what the parent needs
    }
public:
    int diameterOfBinaryTree(TreeNode* root) { depth(root); return best; }
};
```

Two important variants:

**The poison-value trick (LC 110 Balanced).** Return `-1` to mean "already unbalanced below" and propagate it up, so the whole check is one  $O(n)$  pass instead of  $O(n \log n)$ :

```
int h(TreeNode* n) {
    if (!n) return 0;
    int L = h(n->left); if (L == -1) return -1;
    int R = h(n->right); if (R == -1) return -1;
    if (abs(L - R) > 1) return -1;
    return 1 + max(L, R);
}
bool isBalanced(TreeNode* root) { return h(root) != -1; }
```

**Clamping at zero (LC 124 Max Path Sum).** With negative values, a subtree that contributes negatively should be dropped, so clamp its contribution to 0:

```
int best = INT_MIN;
int gain(TreeNode* n) {
    if (!n) return 0;
    int L = max(0, gain(n->left)); // discard negative branches
    int R = max(0, gain(n->right));
    best = max(best, n->val + L + R); // RECORD: path bending at n
    return n->val + max(L, R); // RETURN: one straight branch only
}
```

**Problems:** 543 Diameter, 110 Balanced, 124 Max Path Sum, 687 Longest Univalued Path, 1372 Longest ZigZag, 250 Count Univalued Subtrees, 333 Largest BST Subtree.

## Pattern 4 — Root-to-leaf paths

**Shape:** top-down, carrying the running path or running sum, and recording an answer **only at leaves**. Whenever you push onto a path vector, you must `pop_back()` on the way out — this is backtracking.

```
void dfs(TreeNode* n, int target, vector<int>& path, vector<vector<int>>& out) {
    if (!n) return;
    path.push_back(n->val);
    target -= n->val;
    if (!n->left && !n->right && target == 0) out.push_back(path); // LEAF only
    dfs(n->left, target, path, out);
    dfs(n->right, target, path, out);
    path.pop_back(); // BACKTRACK
}
```

The two things that break this pattern:

- **Testing the sum at a null node instead of a leaf.** `if (!n) { if (target==0) ... }` double-counts, because a leaf has two null children.
- **Forgetting `pop_back()`.** The path vector then accumulates every node ever visited. If instead you pass `path` by value, no pop is needed — simpler to get right, but it copies, so it is slower.

**Problems:** 112 Path Sum, 113 Path Sum II, 257 Binary Tree Paths, 129 Sum Root to Leaf Numbers, 988 Smallest String Starting From Leaf, 1022 Sum of Root To Leaf Binary Numbers.

**Related but different — LC 437 Path Sum III**, where the path may start and end anywhere downward. That one is a prefix-sum-plus-hashmap problem, exactly like subarray-sum-equals-k on an array, run along the DFS path:

```
unordered_map<long,int> cnt{{0,1}};
int dfs(TreeNode* n, long run, int target) {
    if (!n) return 0;
    run += n->val;
    int res = cnt.count(run - target) ? cnt[run - target] : 0;
    cnt[run]++;
    res += dfs(n->left, run, target) + dfs(n->right, run, target);
    cnt[run]--;
    return res;
}
```

## Pattern 5 — The level-order family

**Shape:** the BFS template from Part 2.5, with a small edit inside the level loop. If a problem says *level*, *row*, *view*, *width*, *depth-by-depth*, or *nearest*, reach for this first.

```
while (!q.empty()) {
    int sz = q.size();
    for (int i = 0; i < sz; i++) {
        TreeNode* n = q.front(); q.pop();
        // ---- the edit goes here ----
        if (n->left) q.push(n->left);
        if (n->right) q.push(n->right);
    }
}
```

| EDIT                                                                   | GIVES YOU                         |
|------------------------------------------------------------------------|-----------------------------------|
| collect all, push the vector                                           | 102 Level Order                   |
| collect all, <code>reverse</code> at the end                           | 107 Bottom-Up Level Order         |
| take the last element of the level                                     | 199 Right Side View               |
| reverse alternate levels (or use a deque)                              | 103 Zigzag                        |
| average of the level                                                   | 637 Average of Levels             |
| max of the level                                                       | 515 Largest Value in Each Row     |
| return the depth on the first leaf you dequeue                         | 111 Minimum Depth — best solution |
| index each node <code>i</code> , width = <code>last - first + 1</code> | 662 Maximum Width                 |
| link each dequeued node to <code>q.front()</code> unless it is last    | 116 / 117 Next Right Pointers     |

**Two-line trap for 662 Maximum Width:** index children as `2i` and `2i+1` and the index overflows on a deep skewed tree. Subtract the first index of each level (`i - levelStart`) to keep the numbers small.

## Pattern 6 — Two-tree comparison and structural edits

**Shape:** recurse on **two nodes at once**, in lockstep. The base cases handle the four combinations of null.

```
bool isSame(TreeNode* p, TreeNode* q) {
    if (!p && !q) return true;           // both empty -> equal
    if (!p || !q) return false;         // exactly one -> unequal
    return p->val == q->val
        && isSame(p->left, q->left)
        && isSame(p->right, q->right);
}
```

Mirror the pairing to get symmetry — compare left against right:

```
bool mirror(TreeNode* a, TreeNode* b) {
    if (!a && !b) return true;
    if (!a || !b) return false;
    return a->val == b->val && mirror(a->left, b->right) && mirror(a->right, b->left);
}
bool isSymmetric(TreeNode* root) { return !root || mirror(root->left, root->right); }
```

Structural edits use the same one-tree skeleton with a swap or a merge in the middle:

```
TreeNode* invertTree(TreeNode* root) { // 226
    if (!root) return nullptr;
    swap(root->left, root->right);
    invertTree(root->left);
    invertTree(root->right);
    return root;
}
```

**Problems:** 100 Same Tree, 101 Symmetric Tree, 226 Invert, 617 Merge Two Binary Trees, 572 Subtree of Another Tree, 951 Flip Equivalent Trees.

**572 Subtree of Another Tree** is `isSame` called at every node —  $O(n \cdot m)$ . The optimal answer is serialise both trees with null markers and run a substring search, which is  $O(n+m)$ ; mention it even if you code the simple version.

## Pattern 7 — Lowest common ancestor

In a plain binary tree, the standard solution is four lines and worth memorising exactly:

```
TreeNode* lca(TreeNode* root, TreeNode* p, TreeNode* q) {
    if (!root || root == p || root == q) return root;
    TreeNode* L = lca(root->left, p, q);
    TreeNode* R = lca(root->right, p, q);
    if (L && R) return root;           // p and q split here -> this is the LCA
    return L ? L : R;                 // both on one side -> pass it up
}
```

Read the return value as "a node I found below me, or the answer." If both sides return something, the current node is where the two paths meet.

In a **BST**, you do not need to search at all — the values tell you which way to go.  $O(h)$ , no recursion needed:

```

TreeNode* lcaBST(TreeNode* root, TreeNode* p, TreeNode* q) {
    while (root) {
        if (p->val < root->val && q->val < root->val)    root = root->left;
        else if (p->val > root->val && q->val > root->val) root = root->right;
        else return root;                             // the split point
    }
    return nullptr;
}

```

**Problems:** 236 LCA of a Binary Tree, 235 LCA of a BST, 1650 LCA III (with parent pointers — use the two-pointer intersection trick from linked lists), 1123 LCA of Deepest Leaves, 865 Smallest Subtree with all the Deepest Nodes.

## Pattern 8 — Construction from traversals

**Shape:** one traversal tells you the **root**, the other tells you **how the rest splits**. Build recursively over index ranges, and use a hash map from value to inorder index so each root lookup is O(1) instead of a scan.

```

unordered_map<int,int> pos;                // value -> index in inorder
int pre = 0;

TreeNode* build(vector<int>& preorder, int lo, int hi) {
    if (lo > hi) return nullptr;
    int rootVal = preorder[pre++];         // preorder gives the root...
    TreeNode* n = new TreeNode(rootVal);
    int mid = pos[rootVal];                // ...inorder says where it splits
    n->left = build(preorder, lo, mid - 1);
    n->right = build(preorder, mid + 1, hi);
    return n;
}

```

For **inorder + postorder**, read the postorder array **backwards** and build the **right** subtree first, because the last element of a postorder is the root and the element before it belongs to the right side.

| GIVEN                              | RECONSTRUCTABLE?                                              |
|------------------------------------|---------------------------------------------------------------|
| preorder + inorder                 | yes                                                           |
| postorder + inorder                | yes                                                           |
| preorder + postorder               | only if every node has 0 or 2 children (not unique otherwise) |
| level-order + inorder              | yes                                                           |
| preorder of a <b>BST</b> alone     | yes — inorder is just the sorted preorder                     |
| sorted array → height-balanced BST | yes — take the middle as the root, recurse on halves          |

That last row is LC 108 and it is three lines:

```

TreeNode* build(vector<int>& a, int lo, int hi) {
    if (lo > hi) return nullptr;
    int mid = lo + (hi - lo) / 2;
    TreeNode* n = new TreeNode(a[mid]);
    n->left = build(a, lo, mid - 1);
    n->right = build(a, mid + 1, hi);
    return n;
}

```

**Problems:** 105, 106, 889, 108 Sorted Array to BST, 109 Sorted List to BST, 1008 BST from Preorder.

## Pattern 9 — BST operations

Search, insert and delete are in Part 3.2. The wider family all uses the same "compare and descend" motion, which is why they are all  $O(h)$ :

```

// floor: the largest value <= key
int floorBST(TreeNode* root, int key) {
    int best = INT_MIN;
    while (root) {
        if (root->val == key) return key;
        if (root->val < key) { best = root->val; root = root->right; } // candidate, look right
        else root = root->left;
    }
    return best;
}

```

Ceil is the same with the comparisons flipped. Range sum (LC 938) prunes whole subtrees that cannot contribute:

```

int rangeSumBST(TreeNode* n, int lo, int hi) {
    if (!n) return 0;
    if (n->val < lo) return rangeSumBST(n->right, lo, hi); // whole left subtree is too small
    if (n->val > hi) return rangeSumBST(n->left, lo, hi); // whole right subtree is too big
    return n->val + rangeSumBST(n->left, lo, hi) + rangeSumBST(n->right, lo, hi);
}

```

**Problems:** 700 Search, 701 Insert, 450 Delete, 938 Range Sum, 270 Closest Value, 235 LCA, 653 Two Sum IV, 1382 Balance a BST.

## Pattern 10 — BST plus inorder ordering

Anything asking for order, rank or sortedness in a BST is an inorder traversal in disguise. Keep a [prev](#) pointer while walking and you can compare consecutive values without materialising an array:

```

TreeNode* prev = nullptr;
bool inorderCheck(TreeNode* n) { // validate a BST in O(1) extra space
    if (!n) return true;
    if (!inorderCheck(n->left)) return false;
    if (prev && prev->val >= n->val) return false;
    prev = n;
    return inorderCheck(n->right);
}

```

**LC 99 Recover BST** is the same walk with the inversions recorded. Exactly two nodes are swapped, which produces **one** inversion if they are adjacent in the inorder sequence and **two** if they are not:

```

TreeNode *first = nullptr, *second = nullptr, *prev = nullptr;
void scan(TreeNode* n) {
    if (!n) return;
    scan(n->left);
    if (prev && prev->val > n->val) { if (!first) first = prev; second = n; }
    prev = n;
    scan(n->right);
}
// then swap(first->val, second->val)

```

**Reverse inorder** (right, node, left) visits values in *descending* order, which is what LC 538 / 1038 Convert BST to Greater Tree needs — a running suffix sum.

**Problems:** 98 Validate, 230 Kth Smallest, 99 Recover, 501 Mode in BST, 530 Minimum Absolute Difference, 538/1038 Greater Tree, 173 BST Iterator, 897 Increasing Order Search Tree.

## Pattern 11 — Serialisation and subtree encoding

**Shape:** turn a tree into a string, or a subtree into a key. Preorder with explicit null markers is the standard choice because the null markers make the structure unambiguous.

```

void ser(TreeNode* n, string& s) {
    if (!n) { s += "#,"; return; } // the null marker is essential
    s += to_string(n->val) + ",";
    ser(n->left, s); ser(n->right, s);
}

TreeNode* des(istream& in) {
    string tok; getline(in, tok, ',');
    if (tok == "#") return nullptr;
    TreeNode* n = new TreeNode(stoi(tok));
    n->left = des(in);
    n->right = des(in);
    return n;
}

```

Without the `#` markers, `[1,2]` could mean 2 is either child and the string is not decodable. Say that if asked why the markers matter.

The same encoding, computed **bottom-up** and stored in a hash map, identifies duplicate subtrees (LC 652) and lets you compare subtrees by string equality.

**Problems:** 297 Serialize/Deserialize Binary Tree, 449 Serialize a BST (no markers needed — BST order recovers structure), 652 Find Duplicate Subtrees, 572 Subtree of Another Tree via serialisation.

## Pattern 12 — Treating the tree as a graph

**Shape:** the problem needs to move **upward or sideways**, not only down. Trees have no parent pointers, so build them first, then run an ordinary BFS treating each node as having up to three neighbours: left, right, parent.

```
unordered_map<TreeNode*, TreeNode*> par;
void mark(TreeNode* n, TreeNode* p) {
    if (!n) return;
    par[n] = p;
    mark(n->left, n); mark(n->right, n);
}
// then BFS from the target over {left, right, par[node]} with a visited set
```

This one idea solves an entire class: distance-K nodes (863), burning/infection time (2385), amount of time to inform employees, all-nodes-within-distance problems, and "print all ancestors."

**Problems:** 863 All Nodes Distance K, 1372 (variant), 2385 Amount of Time for Binary Tree to Be Infected, 1650 LCA with parent pointers, 742 Closest Leaf.

## Pattern 13 — In-place restructuring

**Shape:** rewire pointers rather than build a new tree. These problems are where linked-list pointer discipline pays off: *save what you are about to overwrite*.

**LC 114 Flatten to a linked list** — the elegant version uses the rightmost node of the left subtree as the splice point:

```
void flatten(TreeNode* root) {
    TreeNode* cur = root;
    while (cur) {
        if (cur->left) {
            TreeNode* pred = cur->left;
            while (pred->right) pred = pred->right; // rightmost of the left subtree
            pred->right = cur->right; // splice the old right tail after it
            cur->right = cur->left; // move the left subtree across
            cur->left = nullptr; // and cut
        }
        cur = cur->right;
    }
}
```

**LC 116 / 117 Next right pointers** — level order does it in  $O(n)$  space; the  $O(1)$ -space version walks each level using the `next` pointers you built on the level above, assembling the next level with a dummy node, exactly like building a list.

**Problems:** 114 Flatten, 116 / 117 Populating Next Right Pointers, 156 Upside Down Binary Tree, 426 Convert BST to Sorted Doubly Linked List, 897 Increasing Order Search Tree.

## Pattern 14 — Counting and DP on trees

**Shape:** each node returns a **small tuple of alternatives**, and the parent picks the best combination. This is dynamic programming where the "subproblems" are subtrees.

**LC 337 House Robber III** — return `{best if I skip this node, best if I rob it}`:

```

pair<int,int> rob(TreeNode* n) {                // {skip, take}
    if (!n) return {0, 0};
    auto L = rob(n->left), R = rob(n->right);
    int skip = max(L.first, L.second) + max(R.first, R.second);
    int take = n->val + L.first + R.first;      // children must be skipped
    return {skip, take};
}

```

**LC 96 Unique BSTs** is not a traversal at all — it is the Catalan recurrence, because choosing each value as the root splits the remaining values into a fixed left set and right set:

```

int numTrees(int n) {
    vector<int> dp(n + 1, 0);
    dp[0] = dp[1] = 1;
    for (int i = 2; i <= n; i++)
        for (int r = 1; r <= i; r++)
            dp[i] += dp[r - 1] * dp[i - r];
    return dp[n];
}

```

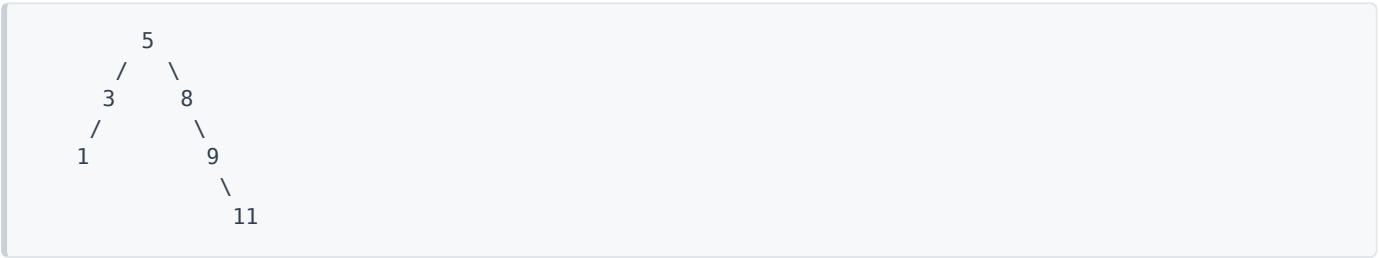
**Problems:** 337 House Robber III, 1448 Count Good Nodes, 96 Unique BSTs, 95 Unique BSTs II, 508 Most Frequent Subtree Sum, 979 Distribute Coins in Binary Tree, 968 Binary Tree Cameras.

# PART 5 — How to approach ANY tree problem

Six steps. Follow them in order and you will rarely be stuck for more than a minute.

## Step 1 — Draw a tree with 5-7 nodes, and make it lopsided

Not a perfect tree. Perfect trees hide the bugs. Draw something with a node that has only one child, and at least one leaf at a different depth:



Every off-by-one and every missing-child bug you are about to write shows up on this shape and not on a symmetric one.

## Step 2 — Name the pattern

Run down the Part 4 table. In practice the keywords do the work:

| THE PROBLEM SAYS                      | REACH FOR                      |
|---------------------------------------|--------------------------------|
| level, row, view, width, nearest      | Pattern 5 — BFS                |
| path, leaf, sum along a path          | Pattern 4 — root-to-leaf DFS   |
| height, depth, diameter, balanced     | Patterns 2 / 3 — postorder     |
| BST, sorted, kth, validate            | Patterns 9 / 10 — inorder      |
| two trees, same, mirror, subtree      | Pattern 6 — lockstep recursion |
| ancestor, common                      | Pattern 7 — LCA                |
| build, construct                      | Pattern 8                      |
| distance, parent, upward              | Pattern 12 — graph conversion  |
| serialize, encode, duplicate subtrees | Pattern 11                     |

## Step 3 — Decide top-down or bottom-up

Ask: "can I answer for this node using only what is below it?"

- **Yes** → bottom-up. Write a function that returns the value; the base case is the identity element for the combining operation.
- **No, I need to know where I came from** → top-down. Pass the state (running sum, path, depth, allowed range) as a parameter.

- **I need both** → Pattern 3: return what the parent needs, record the answer in a captured variable.

## Step 4 — Write the base case first, literally first

Type `if (!node) return X;` before anything else, and be precise about `X`:

| RETURNING                | BASE VALUE                                                      |
|--------------------------|-----------------------------------------------------------------|
| a count or a sum         | <code>0</code>                                                  |
| a height (edges)         | <code>-1</code>                                                 |
| a height (nodes)         | <code>0</code>                                                  |
| a maximum                | <code>INT_MIN</code> , or <code>0</code> if you clamp negatives |
| a boolean "is this fine" | <code>true</code>                                               |
| a node pointer           | <code>nullptr</code>                                            |

Getting `X` right is usually the whole problem; the rest of the function is one line.

## Step 5 — Combine, then check the leaf case separately

Write the combine step. Then ask specifically: *does a leaf behave correctly here?* And: *does a node with exactly one child behave correctly?* Those are the two shapes that break tree code (LC 111 and LC 112 exist purely to catch this).

## Step 6 — Dry-run four inputs before you submit

1. `nullptr` — the empty tree.
2. A single node.
3. Your lopsided tree from Step 1.
4. A skewed stick of 4 nodes — the linked-list shape.

If those four pass on paper, submit. If a problem involves negative values (124, 437, 112), add a fifth: **all values negative**.

# PART 6 — The bug catalogue

---

Ten bugs account for nearly every wrong answer in this topic. When a submission fails, read this list before re-reading your code.

- 1. Missing null base case.** You wrote the leaf check but not `if (!node) return`. Symptom: segmentation fault, or a crash on the very first test.
- 2. Base case at the leaf instead of at null.** Nodes with exactly one child get skipped or double-counted. Symptom: wrong answer only on lopsided trees.
- 3. `min(left, right)` in minimum depth.** A missing child reports a phantom path of length 0. Symptom: LC 111 returns 1 on `[1,2]`. Fix: the single-child special case in Pattern 2.
- 4. Checking a path sum at a null node.** Every leaf has two null children, so the answer is counted twice. Symptom: duplicated paths in LC 113.
- 5. Forgetting `pop_back()` when backtracking.** The path vector keeps growing across branches. Symptom: paths that contain nodes from other branches.
- 6. Reading `q.size()` inside the level loop.** The children you just pushed become part of the current level. Symptom: LC 102 returns one giant level.
- 7. Validating a BST against the parent only.** The classic 10/5/15/6 counterexample from Part 3.1 passes. Fix: the range method, or an inorder scan.
- 8. `INT_MIN` / `INT_MAX` sentinels in BST validation.** A node legitimately holding `INT_MIN` fails a `<=` comparison against the sentinel. Fix: use `long`, or pass `TreeNode*` bounds and null-check them.
- 9. Recomputing height inside another recursion.** `isBalanced` calling `height` at every node, or `diameter` calling `depth`, gives  $O(n^2)$  and times out on large inputs. Fix: Pattern 3.
- 10. Overwriting a pointer you still need.** In flatten / next-pointer problems, `cur->right = cur->left` before saving the old `cur->right` loses the whole right subtree. Same discipline as linked lists: **save `next` before you overwrite it.**

Two more worth naming:

- 11. Comparing node pointers instead of values (or the reverse).** In LC 236 the comparison is by pointer identity; in LC 235 it is by value. Mixing them up passes the samples and fails the hidden tests.
- 12. Not handling `root == nullptr` at the top level.** Almost every LeetCode tree problem includes an empty-tree case. It is free marks and it is skipped constantly.

# PART 7 — Complexity cheat sheet

`n` = number of nodes, `h` = height of the tree, `= log n` when balanced, `= n` when skewed.

| OPERATION                                                | TIME                     | SPACE                                  | NOTE                                    |
|----------------------------------------------------------|--------------------------|----------------------------------------|-----------------------------------------|
| Any full traversal (pre/in/post/level)                   | $O(n)$                   | $O(h)$ recursive, $O(n)$ for BFS queue | every node once                         |
| Morris inorder                                           | $O(n)$                   | <b><math>O(1)</math></b>               | mutates the tree temporarily            |
| Height / size / sum                                      | $O(n)$                   | $O(h)$                                 |                                         |
| Diameter, balanced, max path sum (one pass)              | $O(n)$                   | $O(h)$                                 | Pattern 3                               |
| Diameter or balanced done naively                        | $O(n^2)$                 | $O(h)$                                 | recomputing height inside the recursion |
| BST search / insert / delete / floor / ceil              | <b><math>O(h)</math></b> | $O(1)$ iterative                       | $O(\log n)$ balanced, $O(n)$ skewed     |
| BST k-th smallest                                        | $O(h + k)$               | $O(h)$                                 | early-stopping inorder                  |
| Validate BST                                             | $O(n)$                   | $O(h)$                                 |                                         |
| Build from preorder + inorder                            | $O(n)$ with a hash map   | $O(n)$                                 | $O(n^2)$ without the map                |
| Sorted array → balanced BST                              | $O(n)$                   | $O(\log n)$                            |                                         |
| LCA in a binary tree                                     | $O(n)$                   | $O(h)$                                 |                                         |
| LCA in a BST                                             | $O(h)$                   | $O(1)$ iterative                       |                                         |
| Serialize / deserialize                                  | $O(n)$                   | $O(n)$                                 |                                         |
| Distance-K / graph conversion                            | $O(n)$                   | $O(n)$                                 | parent map + visited set                |
| <code>std::map</code> / <code>std::set</code> operations | $O(\log n)$              | -                                      | red-black tree                          |

**The sentence to say in interviews:** "Time  $O(n)$  because every node is visited once; space  $O(h)$  for the recursion stack, which is  $O(\log n)$  if the tree is balanced and  $O(n)$  in the worst case when it is skewed."

# PART 8 — The problem list

---

Work these in order. Every one is solved in the companion **Trees & BST — Solved Questions** PDF, in exactly this order — but attempt each problem yourself first and only read the solution after you have struggled with it.

## Tier 1 — Learn the mechanics (do all twelve first)

| #   | PROBLEM                           | PATTERN                        |
|-----|-----------------------------------|--------------------------------|
| 94  | Binary Tree Inorder Traversal     | 1 — traversal                  |
| 144 | Binary Tree Preorder Traversal    | 1 — traversal                  |
| 145 | Binary Tree Postorder Traversal   | 1 — traversal                  |
| 102 | Binary Tree Level Order Traversal | 5 — BFS                        |
| 104 | Maximum Depth of Binary Tree      | 2 — aggregate                  |
| 111 | Minimum Depth of Binary Tree      | 2 — aggregate, the trap        |
| 226 | Invert Binary Tree                | 6 — structural edit            |
| 100 | Same Tree                         | 6 — lockstep                   |
| 101 | Symmetric Tree                    | 6 — mirrored lockstep          |
| 112 | Path Sum                          | 4 — root-to-leaf               |
| 543 | Diameter of Binary Tree           | 3 — return one, record another |
| 110 | Balanced Binary Tree              | 3 — poison value               |

Twelve problems. When you can write all twelve from a blank page — recursively **and** the traversals iteratively — you have the mechanics.

## Tier 2 — The placement core (these are what actually get asked)

| #   | PROBLEM                                     | PATTERN                  |
|-----|---------------------------------------------|--------------------------|
| 700 | Search in a Binary Search Tree              | 9 — BST ops              |
| 701 | Insert into a Binary Search Tree            | 9 — BST ops              |
| 450 | Delete Node in a BST                        | 9 — the three cases ★    |
| 98  | Validate Binary Search Tree                 | 10 — range method ★      |
| 230 | Kth Smallest Element in a BST               | 10 — early-stop inorder  |
| 235 | Lowest Common Ancestor of a BST             | 7 — LCA by value         |
| 236 | Lowest Common Ancestor of a Binary Tree     | 7 — LCA ★                |
| 108 | Convert Sorted Array to BST                 | 8 — construction         |
| 105 | Construct Tree from Preorder and Inorder    | 8 — construction ★       |
| 106 | Construct Tree from Inorder and Postorder   | 8 — construction         |
| 103 | Binary Tree Zigzag Level Order              | 5 — BFS variant          |
| 199 | Binary Tree Right Side View                 | 5 — BFS variant          |
| 113 | Path Sum II                                 | 4 — paths + backtracking |
| 129 | Sum Root to Leaf Numbers                    | 4 — paths                |
| 114 | Flatten Binary Tree to Linked List          | 13 — in-place rewiring   |
| 116 | Populating Next Right Pointers in Each Node | 13 — in-place rewiring   |

★ **236, 98, 105 and 450 are the four most-asked tree questions in on-campus drives.**

Amazon, Microsoft, TCS Digital, Infosys, Accenture, Wipro — all of them. Be able to write these four cold.

### Tier 3 — Harder, but they do appear

| #    | PROBLEM                               | PATTERN                 |
|------|---------------------------------------|-------------------------|
| 124  | Binary Tree Maximum Path Sum          | 3 — clamp at zero       |
| 297  | Serialize and Deserialize Binary Tree | 11 — encoding           |
| 437  | Path Sum III                          | 4 + prefix-sum hashmap  |
| 572  | Subtree of Another Tree               | 6 — comparison          |
| 863  | All Nodes Distance K in Binary Tree   | 12 — tree as a graph    |
| 99   | Recover Binary Search Tree            | 10 — inorder inversions |
| 337  | House Robber III                      | 14 — tree DP            |
| 662  | Maximum Width of Binary Tree          | 5 — BFS with indices    |
| 173  | Binary Search Tree Iterator           | 10 — paused inorder     |
| 1448 | Count Good Nodes in Binary Tree       | 14 — top-down counting  |

### The classic Indian-placement questions (GfG / written-round favourites)

These come up in written rounds and technical interviews specifically, often phrased in GfG's wording rather than LeetCode's:

- All four traversals, recursive **and** iterative — asked constantly
- Level-order traversal in spiral / zigzag form
- Height, diameter and mirror of a binary tree
- Left view, right view, **top view**, **bottom view** of a binary tree
- Vertical order traversal and vertical sum
- Check whether two trees are identical / mirror images
- Print all root-to-leaf paths, and all ancestors of a given node
- Lowest common ancestor with and without parent pointers
- Convert a binary tree to its mirror, in place
- Check whether a tree is a BST; find the largest BST inside a binary tree
- Kth smallest and kth largest element in a BST
- Floor and ceil of a key in a BST
- Inorder successor and predecessor of a node in a BST
- Convert a sorted array / sorted linked list into a balanced BST
- Convert a BST into a sorted doubly linked list, in place
- Merge two BSTs into one balanced BST
- Find pairs in a BST that sum to a target
- Diameter, maximum path sum, and maximum sum from leaf to leaf
- Boundary traversal of a binary tree

- Check whether a binary tree is a sum tree / children-sum property
- Serialise and deserialise a binary tree
- Burn a binary tree from a leaf — minimum time
- Count leaf nodes, count full nodes, count nodes in a complete tree

**Top view and bottom view** are near-certain in Indian written rounds and are not on LeetCode. Both are a BFS carrying a horizontal distance `hd` (`hd-1` going left, `hd+1` going right) into a `map<int,int>`: for the top view keep the **first** node seen at each `hd`; for the bottom view keep the **last**. Write both once, they take four lines each.

# PART 9 — A four-week plan

---

Assumes about 1.5 hours a day, and that you have already finished linked lists.

**Week 1 — Recursion and traversals.** Re-read Parts 0 and 1 until "handle this node, trust the children" feels obvious. Build a tree by hand in a `.cpp` file and print all four traversals. Then write each traversal iteratively without looking. Solve Tier 1: 94, 144, 145, 102, 104, 111. Do not touch BSTs this week.

**Week 2 — The recursion patterns.** Finish Tier 1 — 226, 100, 101, 112, 543, 110. Diameter and Balanced are the important two: write the "return one thing, record another" shape from memory on three separate days. Then start Tier 2 with 113, 129, 103, 199.

**Week 3 — BSTs.** 700, 701, 450, 98, 230, 235, 236. Write BST delete from scratch twice on different days — its three cases are the single most common place candidates freeze. Then construction: 108, 105, 106. Say the invariant out loud each time: *preorder gives the root, inorder gives the split*.

**Week 4 — Hardening.** 114, 116, then Tier 3. Finish by re-solving ten Tier 2 problems on a timer, 20 minutes each, on paper, no compiler. Add the GfG-only views — top view and bottom view — since LeetCode never drills them and Indian written rounds do.

**Ongoing:** when a problem defeats you, do not just read the solution. Write down which of the 14 patterns it was and which of the 12 bugs you hit. After thirty problems that log tells you exactly what to drill.

# Quick reference card

---

```

// ===== the node =====
struct TreeNode { int val; TreeNode *left, *right;
                  TreeNode(int x):val(x),left(nullptr),right(nullptr){} };

// ===== the universal skeleton =====
T solve(TreeNode* n) {
    if (!n) return IDENTITY; // 0 / -1 / true / nullptr / INT_MIN
    T L = solve(n->left), R = solve(n->right);
    return combine(L, R, n->val);
}

// ===== traversals =====
void pre (TreeNode* n){ if(!n)return; visit(n); pre(n->left); pre(n->right); }
void in  (TreeNode* n){ if(!n)return; in(n->left); visit(n); in(n->right); }
void post(TreeNode* n){ if(!n)return; post(n->left); post(n->right); visit(n); }

// ===== iterative inorder =====
stack<TreeNode*> st; TreeNode* cur = root;
while (cur || !st.empty()) {
    while (cur) { st.push(cur); cur = cur->left; }
    cur = st.top(); st.pop();
    visit(cur);
    cur = cur->right;
}

// ===== level order (freeze the level size!) =====
queue<TreeNode*> q; q.push(root);
while (!q.empty()) {
    int sz = q.size();
    for (int i = 0; i < sz; i++) {
        TreeNode* n = q.front(); q.pop();
        if (n->left) q.push(n->left);
        if (n->right) q.push(n->right);
    }
}

// ===== return one thing, record another (diameter) =====
int best = 0;
int depth(TreeNode* n) {
    if (!n) return 0;
    int L = depth(n->left), R = depth(n->right);
    best = max(best, L + R); // record
    return 1 + max(L, R); // return
}

// ===== root-to-leaf with backtracking =====
void dfs(TreeNode* n, vector<int>& path) {
    if (!n) return;
    path.push_back(n->val);
    if (!n->left && !n->right) record(path); // LEAF, not null
    dfs(n->left, path); dfs(n->right, path);
    path.pop_back(); // BACKTRACK
}

// ===== LCA, binary tree =====
TreeNode* lca(TreeNode* r, TreeNode* p, TreeNode* q) {
    if (!r || r == p || r == q) return r;
    TreeNode *L = lca(r->left,p,q), *R = lca(r->right,p,q);
    return (L && R) ? r : (L ? L : R);
}

// ===== validate BST (range method) =====
bool valid(TreeNode* n, long lo, long hi) {
    if (!n) return true;
    if (n->val <= lo || n->val >= hi) return false;
    return valid(n->left, lo, n->val) && valid(n->right, n->val, hi);
}

```

```
// ===== BST delete =====
TreeNode* del(TreeNode* r, int key) {
    if (!r) return nullptr;
    if (key < r->val) r->left = del(r->left, key);
    else if (key > r->val) r->right = del(r->right, key);
    else {
        if (!r->left) return r->right;
        if (!r->right) return r->left;
        TreeNode* s = r->right; while (s->left) s = s->left;
        r->val = s->val;
        r->right = del(r->right, s->val);
    }
    return r;
}
```

### The four rules, one last time:

1. Base case is `nullptr`, and its return value is the identity of whatever you are combining.
2. Handle this node; trust the children. Never trace more than one level deep.
3. If the parent needs a different number than the answer, return the parent's number and record the answer.
4. Say the complexity as  **$O(n)$  time,  $O(h)$  space —  $O(\log n)$  balanced,  $O(n)$  skewed.**